ISTA 130

Longer Programs

Docstrings

- You may have to write one on the test.
- Describe what the function does.
- Give the name, type, and meaning of each parameter.
- Give the type and meaning of the return value.

`while` Loop vs. `for` Loop

- A simple for loop:

```
for i in range(10):
    print(i)
```

- The equivalent while loop:

```
i = 0
while i < 10:
    print(i)
    i = i + 1
```

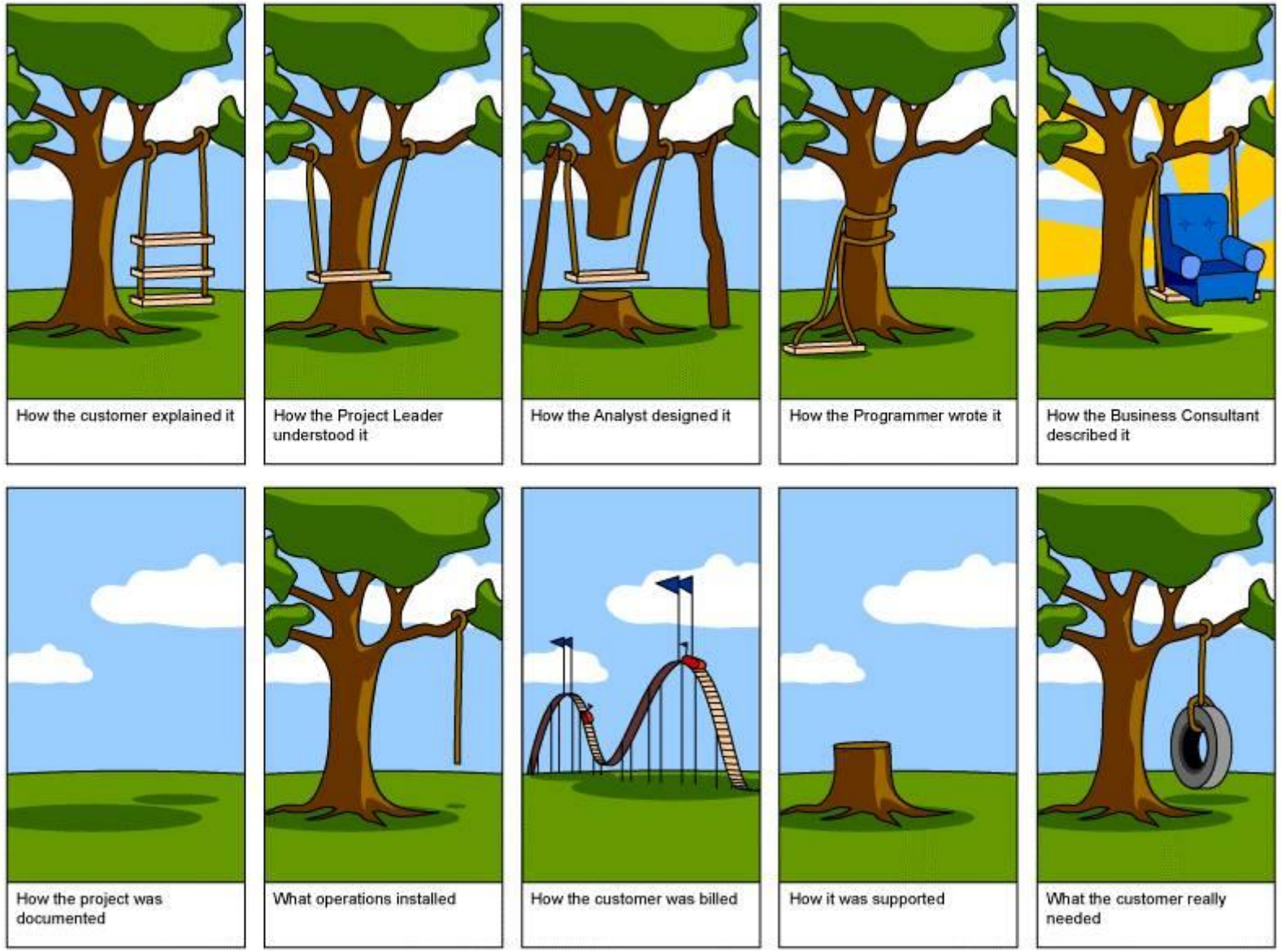
`while True:` and `break`

```
while True:
    # do stuff
    if time_to_quit:
        break
        # sometimes return is also good
```

- An outline of two-person games.
- tic-tac-turtle.

Program Specification (Requirements Documents)

- Our homework specifications are so far very detailed.
  - A requirements document (if you're lucky enough to get one) won't have anywhere near this level of detail).
- Whenever possible, find a way to sit down with some of the users to learn what they need.



Program Specification (Requirements Documents)

Program Specification (Requirements Documents)

- Our homework specifications so far are very detailed
  - a requirements document (if you're lucky enough to get one) won't have anywhere near this level of detail
- How do we approach writing a program that is not as clearly defined?
- Let's walk through an example of writing a program from a somewhat loose specification

Math Tutor

Assume you are asked to write a simple math tutoring program:

- Your program should teach the following topics:
  - odd vs even numbers
  - addition (for integers from 0 through 12)
- Students can choose which topic they wish to work on
- Students can practice either of these two topics for as long as they like.
- Students can switch between the two topics
  - e.g. do a few addition problems, then some odd/even, then back to addition

Program Flow

Here's one way you might have done this:

- get student's name
- present options menu and get choice
- if choice is odd/even problems
  - do odd/even tutoring (until student wants to change)
- otherwise if choice is addition problems
  - do addition tutoring (until student wants to change)
- otherwise if choice is quit, quit
- present menu again and repeat

Functions

- We could write all of the code in `main`, but it would be very complicated and hard to maintain.
- To help manage the complexity we'll break the project down into smaller parts and write functions
  - On scratch paper write out a list of all of the functions you think you'll need
    - write a brief description of what each function will do

Functions

Here's one possibility:

- `practice_odd_even`
  - keep asking odd/even questions until the user wants to switch topics
- `practice_addition`
  - keep asking addition questions until the user wants to switch topics
- `main`
  - present the menu of topic choices, call the chosen function, repeat or quit

Diving Deeper

Let's look into that first function: `practice_odd_even`

- On scratch paper write out the steps this function will involve (in English, not `Python`)
  - Again, try to ignore details for now, just think about the overall flow of the function
- Does the function need any parameters?
- Should it return anything?

Function Flow

Here's one way you might have done this:

`practice_odd_even`

1. generate random integer (perhaps 0 to 100))
2. present integer and ask user for answer
   - if answer is correct print happy message
   - otherwise print sad message
3. ask if user would like to continue with odd/even problems
4. if so, repeat from step 1
5. otherwise exit the loop

Diving Deeper

Let's look into that second function: `practice_addition`

- On scratch paper write out the steps this function will involve (in English, not `Python`)
  - Again, try to ignore details for now, just think about the overall flow of the function
- Does the function need any parameters?
- Should it return anything?

Function Flow

Here's one way you might have done this:

`practice_addition`

1. generate 2 random integers (each from 0 to 12)
2. present addition problem (e.g. $5 + 10$) and ask user for answer
   - if answer is correct print happy message
   - otherwise print sad message
3. ask if user would like to continue with addition problems
4. if so, repeat from step 1
5. otherwise exit the loop

More Functions?

At this point we could probably start writing code, but let's go a bit further first:

- Look back at our function descriptions
- Can you suggest any other functions that might be helpful?

More Functions?

`practice_odd_even`

1. generate random int
2. present integer and ask user for answer
   - if answer is correct print happy message
   - otherwise print sad message
3. ask if user would like to continue
4. if so, repeat from step 1
5. otherwise exit loop

`practice_addition`

1. generate 2 random ints
2. present problem and ask user for answer
   - if answer is correct print happy message
   - otherwise print sad message
3. ask if user would like to continue
4. if so, repeat from step 1
5. otherwise exit loop

More Functions

Here are some possibilities:

- `guess_another`
  - ask the user if he/she would like to keep working on the current topic
- `evaluate_guess`
  - print a happy or sad message depending on student's answer
- Do the proposed functions need any parameters?
- Do they return anything?

Extracting Functions

What if we wrote the original functions before we thought of writing a `guess_another()` function?

- We can always extract common code into a function later.
  - We refer to this as *refactoring*.
- Recall our turtle graphics square function
- Originally we had code like this wherever we needed a square:

```
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.right(90)
```

Extracting Functions

- When we see that we're using the same code in multiple places, we bundle it up into a function (parameterizing literal values if needed):
- ```
  def square(size):
      turtle.forward(size)
      turtle.right(90)
      turtle.forward(size)
      turtle.right(90)
      turtle.forward(size)
      turtle.right(90)
      turtle.forward(size)
      turtle.right(90)
      return None
  ```
- and wherever we had the repeated code we replace it with a call to our function:
- ```
  square(100)
  ```

Extracting Functions

- Similarly, we can put the code that asks the user if he/she would like to continue into a `guess_another()` function and then replace the repeated chunks of code with calls to that function.
- You may not notice that you need a function until you find yourself writing code that feels suspiciously familiar.

Other Things to Think About

As you are designing your solution to a problem there are many things to think about:

- Have you solved similar problems in the past?
- How clear is your solution?
  - Is there a less complicated way to do it?
  - Can you reduce the complexity by defining functions for processes you'll be repeating?
  - Have you used clear and descriptive names for variables and functions?
- Will it be easy to maintain the code?
  - In our math-tutor example can you predict what the customer will request from you next?
    - subtraction problems
    - multiplication problems
    - division problems
  - We should keep that in mind as we design our solution

Writing the Code

- Thinking out your solution before writing code can be very helpful **Design Phase**
- If you write the program steps in English first, the coding will be easier
  - You'll have an outline to follow
  - (You can do this in your code as comments)


0
28
9
26
30
11
7
20
32
17
5
22
34
15
3
24
36
13
1
00
27
10
25
29
12
8
19
31
18
6
21
33
16
4
23
35
14
2
Pasadena, California
Stanford Band